

Computer Organisation & Program Execution 2019



2

Hardware/Software Interface

Uwe R. Zimmer - The Australian National University

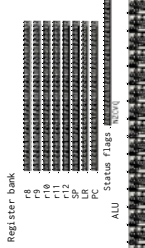
Hardware/Software Interface

References for this chapter

[Patterson17]
David A. Patterson & John L. Hennessy
Computer Organization and Design – The Hardware/Software Interface
Chapter 2 “Instructions: Language of the Computer” & Chapter 3 “Arithmetic for Computers”
AKM edition, Morgan Kaufmann 2017

Hardware/Software Interface

Adding the value of two registers



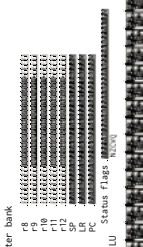
The CPU will fetch the content of the memory cell which PC is pointing to.

⌚ We want the CPU to execute:
 $r4 := r2 + r3$

⌚ What to store in this memory cell?

Hardware/Software Interface

Adding the value of two registers



Op Code Arguments

ANDS <Rd>, <Rn>, <Rn>

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0

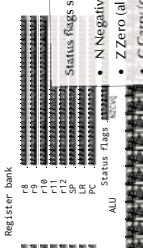
Rn Rd

Disassembler

⌚ We want the CPU to execute:
 $r4 := r2 + r3$

Hardware/Software Interface

Adding the value of two registers



Op Code Arguments

ADDs r4, r2, r3

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

0 0 1 1 0 0 0 1 0 1 0 1 0 1 0 0

Rn Rd

Disassembler

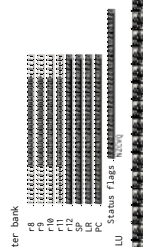
⌚ We want the CPU to execute:
 $r4 := r2 + r3$

Status flags set:

- N Negative (MSB = 1)
- Z Zero (all bits zero)
- C Carry (carry out)
- V Overflow (sign wrong)

Hardware/Software Interface

Adding the value of two registers



Op Code Arguments

ANDS r5, r6

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

0 1 0 0 0 0 0 0 0 1 1 0 1 0 1 1

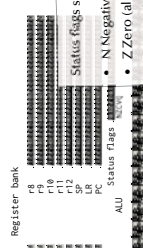
Rn Rd

Disassembler

⌚ We want the CPU to execute:
 $r5 := r5 \& r6$

Hardware/Software Interface

Adding the value of two registers



Op Code Arguments

ADDs r4, r2, r3

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

0 0 1 1 0 0 0 1 0 1 0 1 0 1 0 0

Rn Rd

Disassembler

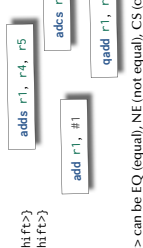
⌚ We want the CPU to execute:
 $r4 := r2 + r3$

Status flags set:

- N Negative (MSB = 1)
- Z Zero (all bits zero)
- C Carry (carry out)
- V Overflow (sign wrong)

Hardware/Software Interface

ARM v7-M 32 bit add instructions



Op Code Arguments

add(s) <Rd>, <Rn>, <Rn>, <#const>

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Rn Rd

Disassembler

⌚ We want the CPU to execute:
 $r4 := r2 + r3$

Status flags set:

- N Negative (MSB = 1)
- Z Zero (all bits zero)
- C Carry (carry out)
- V Overflow (sign wrong)

Hardware/Software Interface

Simple arithmetic inside the CPU

Calculator:

$e := a + b - 2 \times c$

assuming all types are 32bit 2's complement numbers (Integer),
r1 holds a, r2 holds b, r3 holds c, and the results should be in r4.

```
add r5, r1, r2
lsl r6, r3, #1
sub r4, r5, r6
```

We need temporary storage (r5, r6) in the process as we didn't want to overwrite the original values. Yet the total number of registers is always limited.

How about we assume that values are no longer needed after this expression:

Hardware/Software Interface

Simple arithmetic inside the CPU

Calculator:

$e := a + b - 2 \times c$

assuming all types are 32bit 2's complement numbers (Integer),
r1 holds a, r2 holds b, r3 holds c, and the results should be in r4.

```
add r5, r1, r2
lsl r6, r3, #1
sub r4, r5, r6
```

We need temporary storage (r5, r6) in the process as we didn't want to overwrite the original values. Yet the total number of registers is always limited.

How about we assume that values are no longer needed after this expression:

```
add r1, r1, r2
lsl r3, r3, #1
sub r4, r1, r3
```

... your compiler will know when such side-effects are ok and when not.

⚠ Any overflows?

Hardware/Software Interface

Simple arithmetic inside the CPU

Calculator:

$e := a + b - 2 \times c$

We need to check results after each step:

```
adds r1, r1, r2 ; need to check overflow flag
lsl r3, r3, #1 ; need to check that the sign did not change
subs r4, r1, r3 ; need to check overflow flag again
```

⚠ We don't have the means yet to branch off into different actions in case things go bad ... to come soon.

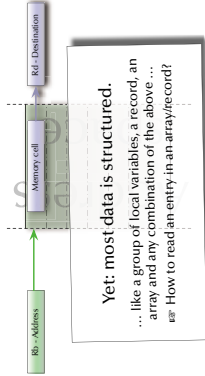
Or we use saturation arithmetic and live with the error:

```
qadd r1, r1, r2
qadd r3, r3, #1
qsub r4, r1, r3
```

⚠ If we know we need to carry on either way, this at least minimizes the local errors.

Hardware/Software Interface

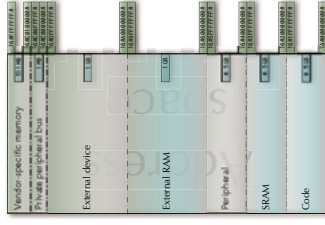
ARM v7-M Copy data in and out of the CPU



In its most basic form the value of a register is interpreted as an **address** and the **memory content** there is loaded into another register.

Hardware/Software Interface

Cortex-M4 Address Space



Your CPU has 32bit of address space

⚠ 4GB

... address space does not equate to physical memory!

... **Not all memory is equal:** Some memory ...

... can be **executed**

... can be **written to or read from or both**

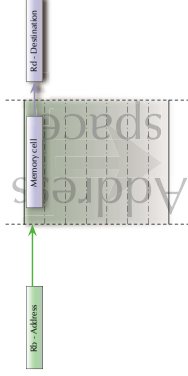
... has **side-effects** (coffee cups fall over)

... has **strictly-ordered** access

... **does not physically exist**

Hardware/Software Interface

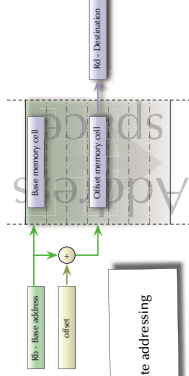
ARM v7-M Copy data in and out of the CPU



In its most basic form the value of a register is interpreted as an **address** and the **memory content** there is loaded into another register.

Hardware/Software Interface

ARM v7-M Move data in and out of the CPU



ldr <Rd> <R0>, [**<R0>**], [**#+/-offset<L>**]]
str <R0> <R0>, [**<R0>**], [**#+/-offset<L>**]]

Reads from a potentially offset memory cell with a base register address.

Hardware/Software Interface

ARM v7-M Move data in and out of the CPU

Immediate addressing ("Pre-indexed")

`ldr<C><Q> <Rd>, [<Rb>, #<#>/-offset>]`
`str<C><Q> <Rs>, [<Rb>, #<#>/-offset>]`

Writes to a potentially offset memory cell with a base register address, and writes the offset address back into the original base register.

`str r1, [r4]`
`str r1, [r4, #12]`

© 2019 Awe K. Zimmer, The Australian National University
 Page 124 of 481 | Chapter 2: "Hardware/Software Interface" up to page 126

Hardware/Software Interface

ARM v7-M Move data in and out of the CPU

Index register addressing ("Post-indexed")

`ldr<C><Q> <Rd>, [<Rb>, #<#>/-offset>]`
`str<C><Q> <Rs>, [<Rb>, #<#>/-offset>]`

Reads from a memory cell with a base register address and writes the offset address back into the original base register.

`ldr r1, [r4, #8]`

© 2019 Awe K. Zimmer, The Australian National University
 Page 127 of 481 | Chapter 2: "Hardware/Software Interface" up to page 126

Hardware/Software Interface

ARM v7-M Move data in and out of the CPU

Index register addressing

`ldr<C><Q> <Rd>, [<Rb>, <Rt>, LSL #<shift>]`
`str<C><Q> <Rs>, [<Rb>, <Rt>, LSL #<shift>]`

Writes to a memory cell with a base register address plus a potentially shifted index register.

`str r1, [r4, r3]`

© 2019 Awe K. Zimmer, The Australian National University
 Page 130 of 481 | Chapter 2: "Hardware/Software Interface" up to page 130

Hardware/Software Interface

ARM v7-M Move data in and out of the CPU

Immediate addressing ("Pre-indexed")

`ldr<C><Q> <Rd>, [<Rb>, #<#>/-offset>]`
`str<C><Q> <Rs>, [<Rb>, #<#>/-offset>]`

Reads from an offset memory cell with a base register address and writes the offset address back into the original base register.

`ldr r1, [r4, #8]`

© 2019 Awe K. Zimmer, The Australian National University
 Page 125 of 481 | Chapter 2: "Hardware/Software Interface" up to page 126

Hardware/Software Interface

ARM v7-M Move data in and out of the CPU

Immediate addressing ("Post-indexed")

`ldr<C><Q> <Rd>, [<Rb>, #<#>/-offset>]`
`str<C><Q> <Rs>, [<Rb>, #<#>/-offset>]`

Writes to a memory cell with a base register address and writes the offset address back into the original base register.

`str r1, [r4, #8]`

© 2019 Awe K. Zimmer, The Australian National University
 Page 128 of 481 | Chapter 2: "Hardware/Software Interface" up to page 126

Hardware/Software Interface

ARM v7-M Move data in and out of the CPU

Literal addressing

`ldr<C><Q> <Rd>, <label>`
`ldr<C><Q> <Rd>, [PC, #<#>/-offset>]`

Reads from a data area embedded into the code section.

Note there is no store version.

`ldr r1, data`

© 2019 Awe K. Zimmer, The Australian National University
 Page 131 of 481 | Chapter 2: "Hardware/Software Interface" up to page 130

Hardware/Software Interface

ARM v7-M Move data in and out of the CPU

Immediate addressing ("Pre-indexed")

`ldr<C><Q> <Rd>, [<Rb>, #<#>/-offset>]`
`str<C><Q> <Rs>, [<Rb>, #<#>/-offset>]`

Writes to an offset memory cell with a base register address and writes the offset address back into the original base register.

`str r1, [r4, #12]`

© 2019 Awe K. Zimmer, The Australian National University
 Page 126 of 481 | Chapter 2: "Hardware/Software Interface" up to page 126

Hardware/Software Interface

ARM v7-M Move data in and out of the CPU

Index register addressing

`ldr<C><Q> <Rd>, [<Rb>, <Rt>, LSL #<shift>]`
`str<C><Q> <Rs>, [<Rb>, <Rt>, LSL #<shift>]`

Reads from a memory cell with a base register address plus a potentially shifted index register.

`ldr r1, [r4, r3]`

© 2019 Awe K. Zimmer, The Australian National University
 Page 129 of 481 | Chapter 2: "Hardware/Software Interface" up to page 126

Hardware/Software Interface

ARM v7-M Move data in and out of the CPU

Multiple registers (positive growing stack)

`stmia<C><Q> <Rb>{<#>, <registers>}`
`ldmia<C><Q> <Rb>{<#>, <registers>}`

Stores "increment after" and loads "decrement before".

`stmia r9, {r1, r3, r4, fp}`

© 2019 Awe K. Zimmer, The Australian National University
 Page 132 of 481 | Chapter 2: "Hardware/Software Interface" up to page 130

Hardware/Software Interface

ARM v7-M Move data in and out of the CPU

Multiple registers (positive growing stack)

Write-back

Relative cell 1

Relative cell n

R0 - Destination

Rn - Destination

R0 - Source

Rn - Source

1dmb SPI, {r1, r3, r4, fp}

stia<C><Q> <R><{>, <R><{>
1dmb<C><Q> <R><{>, <R><{>
Reads multiple registers from sequential memory addresses.
Stores "increment after" and loads "decrement before".

© 2019 Law R. Zimmer, The Australian National University Page 133 of 481 | Chapter 2: "Hardware/Software Interface" up to page 138

Hardware/Software Interface

ARM v7-M Move data in and out of the CPU

Multiple registers (negative growing stack)

Write-back

Relative cell 1

Relative cell n

R0 - Destination

Rn - Destination

R0 - Source

Rn - Source

stmb SPI, {r1, r3, r4, fp}

stmb<C><Q> <R><{>, <R><{>
1dmb<C><Q> <R><{>, <R><{>
Stores multiple registers to sequential memory addresses.
Stores "decrement before" and loads "increment after".

© 2019 Law R. Zimmer, The Australian National University Page 134 of 481 | Chapter 2: "Hardware/Software Interface" up to page 138

Hardware/Software Interface

ARM v7-M Move data in and out of the CPU

Multiple registers (negative growing stack)

Write-back

Relative cell 1

Relative cell n

R0 - Destination

Rn - Destination

R0 - Source

Rn - Source

1dmb SPI, {r1, r3, r4, fp}

stmb<C><Q> <R><{>, <R><{>
1dmb<C><Q> <R><{>, <R><{>
Reads multiple registers from sequential memory addresses.
Stores "decrement before" and loads "increment after".

© 2019 Law R. Zimmer, The Australian National University Page 135 of 481 | Chapter 2: "Hardware/Software Interface" up to page 138

Hardware/Software Interface

Simple arithmetic in memory

Calculate again:

```
e := a + b - 2*c
```

but now a, b, c and e are stored in memory, relative to an address stored in FP ("Frame Pointer"):

a is held at [fp - 12], b at [fp - 16], c at [fp - 20] and e at [fp - 24]

In order to do arithmetic we need to load those values into the CPU

first and afterwards we need to store the result in memory:

```
ldr r1, [fp, #-12]
ldr r2, [fp, #-16]
add r1, r1, r2
ldr r2, [fp, #-20]
lsl r2, r2, #20
sub r1, r1, r2
str r1, [fp, #-24]
```

⚠ Notice that this time we only used two registers.

© 2019 Law R. Zimmer, The Australian National University Page 136 of 481 | Chapter 2: "Hardware/Software Interface" up to page 138

Hardware/Software Interface

Simple arithmetic in memory

Calculate again:

```
e := a + b - 2*c
```

Or in saturation arithmetic:

```
ldr r1, [fp, #-12]
ldr r2, [fp, #-16]
qadd r1, r1, r2
ldr r2, [fp, #-20]
qadd r2, r2, #20
qsub r1, r1, r2
str r1, [fp, #-24]
```

© 2019 Law R. Zimmer, The Australian National University Page 137 of 481 | Chapter 2: "Hardware/Software Interface" up to page 138

Hardware/Software Interface

Simple arithmetic in memory

Calculate again:

```
e := a + b - 2*c
```

Or with overflow checks:

```
ldr r1, [fp, #-12]
ldr r2, [fp, #-16]
adds r1, r1, r2
ldr r2, [fp, #-20]
lsl r2, r2, #20
subs r1, r1, r2
str r1, [fp, #-24]
```

; need to check overflow flag
; need to check that the sign did not change
; need to check overflow flag

⚠ It's time we learn about branching off into alternative execution paths.

© 2019 Law R. Zimmer, The Australian National University Page 138 of 481 | Chapter 2: "Hardware/Software Interface" up to page 138

Hardware/Software Interface

ARM v7-M Branch instructions

<C>	Meanings	Flags
eq	Equal	Z = 1
ne	Not equal	Z = 0
cs, hs	Carry set, Unsigned higher or same	C = 1
cc, lo	Carry clear, Unsigned lower	C = 0
pl	Plus, Positive or zero	N = 0
vs	Overflow	V = 1
vc	No overflow	V = 0
hi	Unsigned higher	C = 1 ∧ Z = 0
ls	Unsigned lower or same	C = 0 ∨ Z = 1
gt	Signed greater	N = 0 ∧ Z = 1
lt	Signed less	N = 1
ge	Signed greater or equal	Z = 0 ∨ N = V
le	Signed less or equal	Z = 1 ∨ N = V
al, <none>	Always	any

© 2019 Law R. Zimmer, The Australian National University Page 139 of 481 | Chapter 2: "Hardware/Software Interface" up to page 138

Hardware/Software Interface

Simple arithmetic in memory

Calculate again:

```
e := a + b - 2*c
```

Or with overflow checks:

```
ldr r1, [fp, #-12]
ldr r2, [fp, #-16]
adds r1, r1, r2
bvs Overflow
ldr r2, [fp, #-20]
adds r2, r2, #20
bvs Overflow
subs r1, r1, r2
bvs Overflow
str r1, [fp, #-24]
```

; branch if overflow is set
; branch if overflow is set
; branch if overflow is set

⚠ Overflow:
svc #5
; call the operating system or runtime environment with #5
; (assuming that #5 indicates an overflow situation)

© 2019 Law R. Zimmer, The Australian National University Page 140 of 481 | Chapter 2: "Hardware/Software Interface" up to page 138

Hardware/Software Interface

Simple arithmetic in memory

Calculate again:

```
e := a + b - 2*c
```

Or with overflow checks:

```
ldr r1, [fp, #-12]
ldr r2, [fp, #-16]
adds r1, r1, r2
bvs Overflow
ldr r2, [fp, #-20]
adds r2, r2, #20
bvs Overflow
subs r1, r1, r2
bvs Overflow
str r1, [fp, #-24]
```

; branch if overflow is set
; branch if overflow is set
; branch if overflow is set

⚠ Overflow:
svc #5
; call the operating system or runtime environment with #5
; (assuming that #5 indicates an overflow situation)

... but how do we know where this happened or how to continue operations?

© 2019 Law R. Zimmer, The Australian National University Page 141 of 481 | Chapter 2: "Hardware/Software Interface" up to page 138

Hardware/Software Interface

Simple arithmetic in memory

Calculate again:
 $e := a * b - 2 * c$
 Or with overflow checks:
 ldr r1, [fb, #12]
 ldr r2, [fb, #16]
 adds r1, r1, r2
 blvs Overflow
 ldr r2, [fb, #20]
 adds r2, r2, r2
 blvs Overflow
 subs r1, r1, r2
 blvs Overflow
 str r1, [fb, #24]
 ...
Overflow:
 ...
 bx lr
 ; ... for example writing a log entry with location
 ; resume operations - assuming the above did not change LR

Hardware/Software Interface

More power

Calculate:
 $c := a * b$
 mov r1, #7
 mov r2, #11
 mov r3, #1
 mov r4, r1
power:
 cbz r2, end_power
 tst r2, #0b1
 beq skip
 mul r3, r4
skip:
 mul r4, r4
 lsr r2, #1
 b power
end_power:
 nop
 ; $c = a * b$
 How many iterations?
 How many cycles?

Hardware/Software Interface

Basic instruction sets

Category	Side effects	ARM v7-M
Arithmetic, Logic	Sets and uses CPU flags	add, adc, qadd, sub, sbc, qsub, rsb, mul, mla, mls, udiv, sdiv, umull, umlal, smull, smlal, and, bic, orr, orn, eor, cmp, cmn, tst, teq
Move and shift registers		mov, lsr, asr, lsl, ror, rrx
Branching	Uses CPU flags	b, bl, bx, blx, tbb, tbh
Load & Store	Effects memory	ldr, str, ldmb, ldmia, stmia, stmb

Hardware/Software Interface

ARM v7-M Essential multiplications and divisions

32bit to 32bit
 mul(s)<><q> <rd, >, <rn>, <rm>
 mla<c> <rd, >, <rn>, <rm>, <ra>
 mla<c> <rd, >, <rn>, <rm>, <ra>
 udiv<c> <rd, >, <rn>, <rm>
 sdiv<c> <rd, >, <rn>, <rm>
 32bit to 64bit
 umull<c> <rdlo, >, <rdhi>, <rn>, <rm>
 umlal<c><q> <rdlo, >, <rdhi>, <rn>, <rm>
 smull<c> <rdlo, >, <rdhi>, <rn>, <rm>
 smlal<c> <rdlo, >, <rdhi>, <rn>, <rm>
 ... versions for narrower numbers, as well as versions which operate on multiple narrower numbers in parallel exist as well.

Hardware/Software Interface

Table based branching

tbb<c><q> <rdn, >, <rn>
 tbn<c><q> <rdn, >, <rn>, lsl #1
 Common usage for byte (8bit) tables
 tbb [PC, R1]
Branch_Table:
 .byte (Case_A - Branch_Table)/2 ; Case_A 8 bit offset
 .byte (Case_B - Branch_Table)/2 ; Case_B 8 bit offset
 .byte (Case_C - Branch_Table)/2 ; Case_C 8 bit offset
 .byte 0x00 ; padding to re-align with halfword boundaries
Case_A:
 ... b End_Case
 ... "break out"
Case_B:
 ... b End_Case
 ... any instruction sequence
 ... "break out"
Case_C:
 ... b End_Case
 ... any instruction sequence
End_Case:
 ... any instruction sequence

Hardware/Software Interface

Basic instruction sets

Category	Side effects	ARM v7-M
Arithmetic, Logic	Sets and uses CPU flags	add, adc, qadd, sub, sbc, qsub, rsb, mul, mla, mls, udiv, sdiv, umull, umlal, smull, smlal, and, bic, orr, orn, eor, cmp, cmn, tst, teq
Move and shift registers		mov, lsr, asr, lsl, ror, rrx
Branching	Uses CPU flags	b, bl, bx, blx, tbb, tbh
Load & Store	Effects memory	ldr, str, ldmb, ldmia, stmia, stmb

Instruction sets in the field:
RISC: Power, ARM, MIPS, Alpha, SPARC, AVR, PIC, ...
CISC: x86, Z80, 6502, 68000, ...

Hardware/Software Interface

Straight power

Calculator:
 $c := a * b$
 mov r1, #7
 mov r2, #11
 mov r3, #1
power:
 cbz r2, end_power
 mul r3, r1
 sub r2, #1
 b power
end_power:
 nop
 ; $c = a * b$
 How many iterations?
 How many cycles?

Hardware/Software Interface

Table based branching

tbb<c><q> <rdn, >, <rn>
 tbn<c><q> <rdn, >, <rn>, lsl #1
 Common usage for halfword (16bit) tables
 tbb [PC, R1, lsl #1]
Branch_Table:
 .hword (Case_A - Branch_Table)/2 ; Case_A 16 bit offset
 .hword (Case_B - Branch_Table)/2 ; Case_B 16 bit offset
 .hword (Case_C - Branch_Table)/2 ; Case_C 16 bit offset
Case_A:
 ... any instruction sequence
 ... "break out"
Case_B:
 ... any instruction sequence
 ... "break out"
Case_C:
 ... any instruction sequence
End_Case:
 ... any instruction sequence

Hardware/Software Interface

Basic instruction sets

Category	Side effects	ARM v7-M
Arithmetic, Logic	Sets and uses CPU flags	add, adc, qadd, sub, sbc, qsub, rsb, mul, mla, mls, udiv, sdiv, umull, umlal, smull, smlal, and, bic, orr, orn, eor, cmp, cmn, tst, teq
Move and shift registers		mov, lsr, asr, lsl, ror, rrx
Branching	Uses CPU flags	b, bl, bx, blx, tbb, tbh
Load & Store	Effects memory	ldr, str, ldmb, ldmia, stmia, stmb

What's missing?
 # Changing CPU privileges and handling interrupts.
 # Synchronizing instructions
 Coming in later chapters about concurrency and operating systems.



Hardware/Software Interface

Summary

Hardware/Software Interface

- **Instruction formats**
 - Register sets
 - Instruction encoding
- **Arithmetic / Logic instructions inside the CPU**
 - Summation, Subtraction, Multiplication, Division
 - Logic and shift operations
- **Load / Store and addressing modes**
 - Direct, relative, indexed, and auto-increment addressing forms
- **Branching**
 - Conditional branching and unconditional jumps.